

Collaborative Discussion 2: Strategy Pattern

Strategy Pattern

The original PaymentProcessor works for a small number of payment methods, but the design becomes harder to maintain as more options are added. Every payment type is handled within the same if/elif structure, therefore adding another method requires modifying the existing class. This conflicts with the Open/Closed Principle, which promotes extending software behaviour without repeatedly modifying existing code (Martin, 2003). The current design also creates tight coupling because PaymentProcessor needs to know the details of every payment method.

In my refactoring, I use the Strategy Pattern to move each payment behaviour into a separate class that implements the same Payment_Strategy interface. PaymentProcessor keeps a reference to the selected strategy and delegates the payment operation to it. Sarcar (2022) explains that the Strategy Pattern separates alternative behaviours into interchangeable strategy implementations. In this implementation, the payment method can also be changed at runtime through change_payment_method() without modifying the processor itself.

```
from abc import ABC, abstractmethod

# Strategy interface
class Payment_Strategy(ABC):

    @abstractmethod
    def pay(self, amount):
        pass
```

```
# Concrete strategies
class CreditCard_Payment(Payment_Strategy):

    def pay(self, amount):
        return f"Credit card payment completed: ${amount:.2f}"

class PayPal_Payment(Payment_Strategy):

    def pay(self, amount):
        return f"PayPal payment completed: ${amount:.2f}"

class BankTransfer_Payment(Payment_Strategy):

    def pay(self, amount):
        return f"Bank transfer started: ${amount:.2f}"

# Context
class PaymentProcessor:

    def __init__(self, strategy):
        self.strategy = strategy

    def change_payment_method(self, strategy):
        self.strategy = strategy

    def process_payment(self, amount):

        # validation for all payment methods
        if amount <= 0:
            raise ValueError("Enter an amount greater than $0.")

        print(self.strategy.pay(amount))

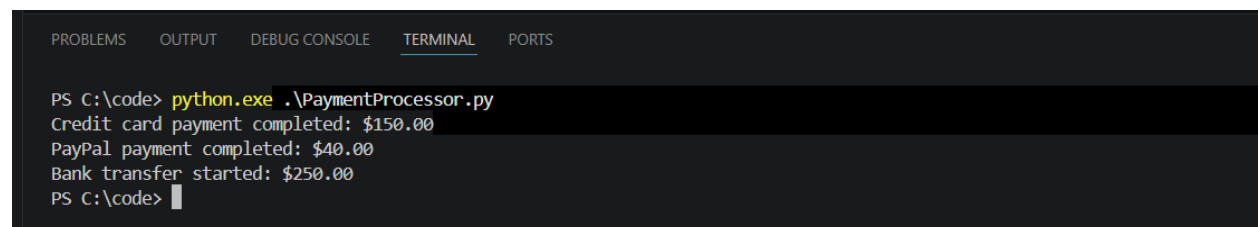
# Usage
processor = PaymentProcessor(CreditCard_Payment())
processor.process_payment(150)

processor.change_payment_method(PayPal_Payment())
processor.process_payment(40)

processor.change_payment_method(BankTransfer_Payment())
processor.process_payment(250)
```

The result:

The implementation successfully processed the three payment methods through the same `PaymentProcessor`. The selected payment method was changed from credit card to PayPal and then to bank transfer using `change_payment_method()`, without modifying the processor itself.

A screenshot of a terminal window with a dark background. At the top, there are tabs labeled 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is selected and underlined), and 'PORTS'. The terminal shows the following text:

```
PS C:\code> python.exe .\PaymentProcessor.py
Credit card payment completed: $150.00
PayPal payment completed: $40.00
Bank transfer started: $250.00
PS C:\code> |
```

This approach makes the design more flexible, readable and easier to test because each payment method is handled by a separate strategy. Shared validation, such as checking that the amount is greater than zero, can remain in `PaymentProcessor` because it applies to all transactions. New payment methods can also be added through new strategy classes without modifying the existing processor. However, the Strategy Pattern introduces additional classes, so it is more suitable when payment behaviours are expected to grow or change rather than in a small, fixed system.

References

Martin, R.C. (2003) *Agile Software Development: Principles, Patterns, and Practices*. Upper Saddle River, NJ: Prentice Hall.

Sarcar, V. (2022) *Java Design Patterns: A Hands-On Experience with Real-World Examples*. 3rd edn. Berkeley, CA: Apress.